

Aggregations in PostgreSQL: GROUP BY, HAVING & Aggregate Functions

Table of Contents

1. [Learning Objectives](#)
 2. [Theory: Aggregation Fundamentals](#)
 3. [Aggregate Functions](#)
 4. [GROUP BY](#)
 5. [HAVING](#)
 6. [COUNT and COUNT DISTINCT](#)
 7. [SUM and AVG](#)
 8. [MIN and MAX](#)
 9. [Aggregation with JOINS](#)
 10. [Filtering with HAVING vs WHERE](#)
 11. [ASCII Visual Diagrams](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Hands-on Exercises](#)
 17. [Advanced Notes](#)
 18. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Use all standard aggregate functions: COUNT, SUM, AVG, MIN, MAX
 - Write GROUP BY queries to summarize data at any granularity
 - Use HAVING to filter aggregate results
 - Distinguish COUNT(*) from COUNT(col) from COUNT(DISTINCT col)
 - Avoid the most common aggregation mistakes
 - Answer aggregation interview questions confidently
-

Theory: Aggregation Fundamentals

Aggregation collapses multiple rows into a single summary row. The query engine processes aggregation in this logical order:

Logical Query Processing Order:

1. FROM – identify source tables
2. JOIN – combine tables
3. WHERE – filter individual rows
4. GROUP BY – form groups
5. HAVING – filter groups
6. SELECT – compute output expressions

7. DISTINCT – remove duplicates
8. ORDER BY – sort
9. LIMIT/OFFSET – paginate

This order explains why:

- You **cannot** use a SELECT alias in WHERE or GROUP BY (it hasn't been computed yet)
- You **can** use aggregate functions in HAVING but not in WHERE
- You **must** GROUP BY every non-aggregated column in SELECT

Setup Tables

```
-- Reuse schema from 01_joins_complete.md or run:
CREATE TABLE IF NOT EXISTS sales (
  sale_id      SERIAL PRIMARY KEY,
  rep_name     VARCHAR(50),
  region       VARCHAR(30),
  product      VARCHAR(50),
  sale_date    DATE,
  amount       NUMERIC(10,2),
  units_sold   INT
);

INSERT INTO sales (rep_name, region, product, sale_date, amount, units_sold) VALUES
('Alice', 'North', 'Widget A', '2024-01-10', 1500.00, 10),
('Alice', 'North', 'Widget B', '2024-01-15', 800.00, 5),
('Bob', 'South', 'Widget A', '2024-01-20', 2200.00, 15),
('Bob', 'South', 'Widget C', '2024-02-05', 1100.00, 8),
('Carol', 'East', 'Widget B', '2024-02-10', 950.00, 7),
('Carol', 'East', 'Widget A', '2024-02-15', 3000.00, 20),
('Alice', 'North', 'Widget A', '2024-03-01', 1800.00, 12),
('Bob', 'South', 'Widget B', '2024-03-10', 600.00, 4),
('Carol', 'East', 'Widget C', '2024-03-15', 1400.00, 10),
('Dave', 'West', 'Widget A', '2024-03-20', 2500.00, 18),
('Alice', NULL, 'Widget D', '2024-04-01', 300.00, 2),
('Eve', 'North', 'Widget A', '2024-04-10', 1700.00, 11);
```

Aggregate Functions

Quick Reference

Function	Description	NULL handling
COUNT(*)	Count all rows in group	Counts NULLs
COUNT(col)	Count non-NULL values	Ignores NULLs
COUNT(DISTINCT col)	Count unique non-NULL values	Ignores NULLs
SUM(col)	Total of numeric values	Ignores NULLs
AVG(col)	Arithmetic mean	Ignores NULLs
MIN(col)	Minimum value	Ignores NULLs
MAX(col)	Maximum value	Ignores NULLs

STRING_AGG(col,sep)	Concatenate strings	Ignores NULLs
ARRAY_AGG(col)	Collect values into array	NULLs included
BOOL_AND(col)	True if all values are true	Ignores NULLs
BOOL_OR(col)	True if any value is true	Ignores NULLs

GROUP BY

GROUP BY partitions rows into groups. One output row is produced per unique combination of GROUP BY columns.

ASCII Diagram — GROUP BY Mechanics

Raw Sales Table:			
rep	region	product	amount
Alice	North	Widget A	1500
Alice	North	Widget B	800
Bob	South	Widget A	2200
Bob	South	Widget C	1100
Carol	East	Widget B	950

GROUP BY rep:	
rep	Group contents
Alice	rows 1,2 → SUM=2300, COUNT=2
Bob	rows 3,4 → SUM=3300, COUNT=2
Carol	rows 5 → SUM= 950, COUNT=1

Examples

```
-- Example 1: Simple GROUP BY – total sales per rep
SELECT
    rep_name,
    COUNT(*)           AS total_sales,
    SUM(amount)        AS total_revenue,
    AVG(amount)        AS avg_sale
FROM sales
GROUP BY rep_name
ORDER BY total_revenue DESC;

-- Example 2: GROUP BY multiple columns
SELECT
    rep_name,
    region,
    SUM(amount)  AS revenue,
    SUM(units_sold) AS units
```

```

FROM sales
GROUP BY rep_name, region
ORDER BY rep_name, region;

-- Example 3: GROUP BY with expression
SELECT
    DATE_TRUNC('month', sale_date) AS month,
    region,
    SUM(amount) AS monthly_revenue
FROM sales
GROUP BY DATE_TRUNC('month', sale_date), region
ORDER BY month, region;

-- Example 4: GROUP BY with FILTER clause (PostgreSQL extension)
SELECT
    region,
    COUNT(*) FILTER (WHERE amount > 1000) AS big_sales,
    COUNT(*) FILTER (WHERE amount <= 1000) AS small_sales,
    COUNT(*) AS total_sales
FROM sales
GROUP BY region;

-- Example 5: Pivot-style aggregation with FILTER
SELECT
    rep_name,
    SUM(amount) FILTER (WHERE product = 'Widget A') AS widget_a_rev,
    SUM(amount) FILTER (WHERE product = 'Widget B') AS widget_b_rev,
    SUM(amount) FILTER (WHERE product = 'Widget C') AS widget_c_rev,
    SUM(amount) AS total_rev
FROM sales
GROUP BY rep_name
ORDER BY total_rev DESC;

```

HAVING

HAVING filters groups **after** aggregation. It is the WHERE clause for aggregate results.

ASCII Diagram — WHERE vs HAVING

Data Flow:

```

FROM sales (12 rows)
  |
  ▼
WHERE (filters rows BEFORE grouping)
  e.g., WHERE region = 'North' → 4 rows
  |
  ▼
GROUP BY rep_name
  Produces N groups

```

|
▼
HAVING (filters groups AFTER aggregation)
e.g., HAVING SUM(amount) > 2000
|
▼
SELECT, ORDER BY, LIMIT

Examples

```
-- Example 6: HAVING with SUM
SELECT
    rep_name,
    SUM(amount) AS total_revenue
FROM sales
GROUP BY rep_name
HAVING SUM(amount) > 2000
ORDER BY total_revenue DESC;

-- Example 7: HAVING with COUNT
SELECT
    product,
    COUNT(*) AS sale_count
FROM sales
GROUP BY product
HAVING COUNT(*) >= 3;

-- Example 8: HAVING with AVG
SELECT
    region,
    ROUND(AVG(amount), 2) AS avg_sale,
    COUNT(*) AS num_sales
FROM sales
GROUP BY region
HAVING AVG(amount) > 1200;

-- Example 9: HAVING vs WHERE – different semantics
-- WHERE filters before grouping (more efficient when possible)
SELECT rep_name, SUM(amount) AS revenue
FROM sales
WHERE sale_date >= '2024-02-01' -- filters rows FIRST
GROUP BY rep_name
HAVING SUM(amount) > 1000; -- then filters groups

-- Example 10: HAVING with multiple conditions
SELECT
    region,
    SUM(amount) AS revenue,
    COUNT(*) AS sales_count
FROM sales
GROUP BY region
```

```
HAVING SUM(amount) > 3000
AND COUNT(*) >= 3;
```

COUNT and COUNT DISTINCT

```
-- Example 11: COUNT variants compared
SELECT
    COUNT(*)                AS total_rows,
    COUNT(region)           AS non_null_regions,
    COUNT(DISTINCT region)  AS unique_regions,
    COUNT(DISTINCT rep_name) AS unique_reps
FROM sales;

-- Example 12: COUNT(*) vs COUNT(col) with NULLs
-- Alice has one sale with NULL region
SELECT
    rep_name,
    COUNT(*)      AS total_sales,
    COUNT(region) AS sales_with_region -- excludes NULL
FROM sales
WHERE rep_name = 'Alice'
GROUP BY rep_name;

-- Example 13: Counting distinct products per region
SELECT
    region,
    COUNT(DISTINCT product) AS unique_products,
    COUNT(*)                AS total_sales
FROM sales
GROUP BY region
ORDER BY unique_products DESC;

-- Example 14: Frequency distribution
SELECT
    units_sold,
    COUNT(*) AS frequency
FROM sales
GROUP BY units_sold
ORDER BY units_sold;
```

SUM and AVG

```
-- Example 15: Running totals and percentages
SELECT
    region,
    SUM(amount)                AS region_revenue,
    ROUND(100.0 * SUM(amount) / SUM(SUM(amount)) OVER (), 2) AS pct_of_total
FROM sales
```

```

GROUP BY region
ORDER BY region_revenue DESC;

-- Example 16: Average with NULLIF to avoid division by zero
SELECT
    product,
    SUM(amount) AS total_revenue,
    SUM(units_sold) AS total_units,
    ROUND(SUM(amount) / NULLIF(SUM(units_sold),0), 2) AS revenue_per_unit
FROM sales
GROUP BY product
ORDER BY revenue_per_unit DESC NULLS LAST;

-- Example 17: Weighted average
SELECT
    region,
    SUM(amount * units_sold) / NULLIF(SUM(units_sold),0) AS weighted_avg_price
FROM sales
GROUP BY region;

```

MIN and MAX

```

-- Example 18: MIN and MAX per group
SELECT
    rep_name,
    MIN(amount) AS smallest_sale,
    MAX(amount) AS largest_sale,
    MAX(amount) - MIN(amount) AS range_amount,
    MIN(sale_date) AS first_sale_date,
    MAX(sale_date) AS last_sale_date
FROM sales
GROUP BY rep_name
ORDER BY largest_sale DESC;

-- Example 19: Find rep with the single largest sale
SELECT rep_name, amount, product
FROM sales
WHERE amount = (SELECT MAX(amount) FROM sales);

```

Aggregation with JOINS

```

-- Example 20: Aggregation after JOIN
SELECT
    c.customer_name,
    COUNT(o.order_id) AS order_count,
    COALESCE(SUM(o.total_amount),0) AS total_spent,
    MAX(o.order_date) AS last_order
FROM customers c

```

```
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spent DESC;
```

Filtering with HAVING vs WHERE

Clause	When it filters
WHERE	Before GROUP BY – filters individual rows; cannot use aggregate functions
HAVING	After GROUP BY – filters groups; can use aggregate functions

```
-- Best practice: use WHERE for row filters, HAVING for group filters
SELECT region, SUM(amount) AS revenue
FROM sales
WHERE sale_date BETWEEN '2024-01-01' AND '2024-12-31' -- row filter
GROUP BY region
HAVING SUM(amount) > 2000; -- group filter
```

Common Mistakes

Mistake 1: Non-aggregated column not in GROUP BY

```
-- WRONG: product is not in GROUP BY or aggregated
SELECT rep_name, product, SUM(amount) FROM sales GROUP BY rep_name;

-- CORRECT:
SELECT rep_name, product, SUM(amount) FROM sales GROUP BY rep_name, product;
```

Mistake 2: Using WHERE with aggregate functions

```
-- WRONG:
SELECT rep_name, SUM(amount) FROM sales WHERE SUM(amount) > 2000 GROUP BY rep_name;

-- CORRECT:
SELECT rep_name, SUM(amount) FROM sales GROUP BY rep_name HAVING SUM(amount) > 2000;
```

Mistake 3: COUNT(*) vs COUNT(col) confusion

```
-- COUNT(*) counts all rows including NULLs
-- COUNT(col) counts only non-NULL values in that column
```



```
SELECT COUNT(*), COUNT(region) FROM sales;
-- COUNT(*) = 12, COUNT(region) = 11 (one NULL region)
```

Mistake 4: AVG of NULLs gives wrong intuition

```
-- AVG ignores NULLs – mean is computed over non-NULL rows only
-- If you want NULL treated as 0:
SELECT AVG(COALESCE(amount, 0)) FROM sales;
```

Best Practices

1. Always use WHERE for non-aggregate filters — it reduces rows before grouping (cheaper).
2. Use HAVING only for conditions that require aggregate values.
3. Qualify GROUP BY columns with table alias when joining.
4. Use COUNT(DISTINCT col) carefully — it is more expensive than COUNT(*).
5. Use the FILTER clause for conditional aggregation instead of CASE inside SUM.
6. Handle NULLs explicitly with COALESCE in SUM/AVG when 0 is semantically correct.
7. When ordering by an aggregate, repeat it in ORDER BY (or use its ordinal position).

Performance Considerations

```
-- 1. An index on the GROUP BY column speeds up grouping
CREATE INDEX idx_sales_region ON sales(region);
CREATE INDEX idx_sales_rep ON sales(rep_name);

-- 2. For time-based GROUP BY, a BRIN index is efficient on large tables
CREATE INDEX idx_sales_date_brin ON sales USING BRIN(sale_date);

-- 3. Partial aggregation (push-down) – filter before grouping
EXPLAIN ANALYZE
SELECT region, SUM(amount)
FROM sales
WHERE sale_date >= '2024-03-01' -- reduces rows first
GROUP BY region;

-- 4. Materialized views for expensive recurring aggregations
CREATE MATERIALIZED VIEW monthly_sales_summary AS
SELECT
    DATE_TRUNC('month', sale_date) AS month,
    region,
    SUM(amount) AS revenue,
    COUNT(*) AS sales_count
FROM sales
GROUP BY 1, 2;

CREATE UNIQUE INDEX ON monthly_sales_summary(month, region);
```

```
-- Refresh when data changes:  
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_sales_summary;
```

Interview Questions & Answers

Q1: What is the difference between WHERE and HAVING?

A: WHERE filters individual rows before grouping and cannot reference aggregate functions. HAVING filters groups after aggregation and can reference aggregate functions. Use WHERE to reduce input rows for efficiency, HAVING to filter on aggregated results.

Q2: What does COUNT(*) count vs COUNT(column)?

A: COUNT() counts every row in the group including rows with NULL values. COUNT(column) counts only rows where that specific column is non-NULL. Use COUNT() for row counts, COUNT(col) when you specifically want to count non-NULL occurrences of a column.

Q3: Can you use an alias defined in SELECT inside HAVING?

A: In standard SQL (and PostgreSQL), you cannot reference a SELECT alias in HAVING because HAVING is evaluated before SELECT. You must repeat the aggregate expression. However, PostgreSQL does allow aliases in ORDER BY.

Q4: How do you count unique values?

A: Use COUNT(DISTINCT column). It counts non-NULL unique values. Note it is more expensive than COUNT(*) as it requires deduplication.

Q5: How do you perform conditional aggregation?

A: Use CASE inside an aggregate: `SUM(CASE WHEN condition THEN amount ELSE 0 END)`. PostgreSQL also supports the cleaner FILTER syntax: `SUM(amount) FILTER (WHERE condition)`.

Q6: What happens to NULLs in aggregate functions?

A: All aggregate functions (SUM, AVG, MIN, MAX, COUNT(col)) ignore NULLs — they are excluded from the computation. Only COUNT(*) includes NULLs (counting the row, not the value).

Q7: How do you calculate the percentage share of each group?

A: Use a window function over the aggregate: `100.0 * SUM(amount) / SUM(SUM(amount)) OVER ()`. The inner SUM aggregates per group, the outer SUM with OVER() computes the grand total.

Q8: What is the FILTER clause?

A: The FILTER clause is a PostgreSQL extension that allows conditional aggregation inline: `COUNT(*) FILTER (WHERE status = 'active')`. It is functionally equivalent to `SUM(CASE WHEN status='active' THEN 1 ELSE 0 END)` but more readable.

Hands-on Exercises

Exercise 1: Sales Leaderboard

Show each sales rep's total revenue, number of sales, and average sale amount. Only include reps with more than 2 sales.

```
-- Solution:
SELECT
    rep_name,
    COUNT(*)           AS num_sales,
    SUM(amount)        AS total_revenue,
    ROUND(AVG(amount), 2) AS avg_sale
FROM sales
GROUP BY rep_name
HAVING COUNT(*) > 2
ORDER BY total_revenue DESC;
```

Exercise 2: Monthly Revenue Trend

Show total revenue for each month in 2024, ordered chronologically.

```
-- Solution:
SELECT
    TO_CHAR(DATE_TRUNC('month', sale_date), 'YYYY-MM') AS month,
    SUM(amount) AS revenue,
    COUNT(*) AS sales_count
FROM sales
WHERE sale_date >= '2024-01-01' AND sale_date < '2025-01-01'
GROUP BY DATE_TRUNC('month', sale_date)
ORDER BY DATE_TRUNC('month', sale_date);
```

Exercise 3: Product Performance

For each product, show total units sold, total revenue, and revenue per unit. Exclude products with fewer than 3 sales.

```
-- Solution:
SELECT
    product,
    SUM(units_sold) AS total_units,
    SUM(amount) AS total_revenue,
    ROUND(SUM(amount) / NULLIF(SUM(units_sold), 0), 2) AS rev_per_unit
FROM sales
GROUP BY product
HAVING COUNT(*) >= 3
ORDER BY total_revenue DESC;
```

Exercise 4: Region Comparison

Show region, total revenue, and what percentage of overall revenue each region contributes. Round to 2 decimal places.

```
-- Solution using subquery:
SELECT
    region,
    SUM(amount) AS revenue,
```

```

ROUND(100.0 * SUM(amount) /
      (SELECT SUM(amount) FROM sales), 2)          AS pct_of_total
FROM sales
WHERE region IS NOT NULL
GROUP BY region
ORDER BY revenue DESC;

```

Exercise 5: Conditional Aggregation

Show each rep's revenue broken down by product (Widget A, B, C as separate columns), plus total.

```

-- Solution:
SELECT
  rep_name,
  SUM(amount) FILTER (WHERE product = 'Widget A') AS widget_a,
  SUM(amount) FILTER (WHERE product = 'Widget B') AS widget_b,
  SUM(amount) FILTER (WHERE product = 'Widget C') AS widget_c,
  SUM(amount)                                     AS total
FROM sales
GROUP BY rep_name
ORDER BY total DESC;

```

Advanced Notes

GROUP BY Ordinal Position

PostgreSQL allows referencing SELECT column position in GROUP BY:

```

SELECT region, SUM(amount) FROM sales GROUP BY 1; -- 1 = region
-- Useful for long expressions but harms readability

```

GROUPING SETS Preview

For multi-level aggregation without UNION ALL, see `07_grouping_sets_rollup_cube.md` :

```

SELECT region, product, SUM(amount)
FROM sales
GROUP BY GROUPING SETS ((region, product), (region), ());
-- Produces subtotals and grand total in one query

```

String Aggregation

```

SELECT
  region,
  STRING_AGG(DISTINCT rep_name, ' ' ORDER BY rep_name) AS reps
FROM sales
WHERE region IS NOT NULL
GROUP BY region;

```

ARRAY_AGG

```
SELECT
    rep_name,
    ARRAY_AGG(DISTINCT product ORDER BY product) AS products_sold
FROM sales
GROUP BY rep_name;
```

Cross-references

- [01_joins_complete.md](#) — JOINS that feed data into aggregations
- [07_grouping_sets_rollup_cube.md](#) — advanced multi-level GROUP BY
- [01_window_functions_intro.md](#) (04_Advanced_SQL) — aggregation without collapsing rows
- [08_Query_Optimization/](#) — index strategies for GROUP BY performance